

Beyond Chat

Team Name:

Team Members:

Kushagra Bharti

Yuvraj Kashyap

Harshwardhan Kothari

Nishant Bhagat

Diya Mehta

Supervisor:

Professor Sridhar Alagar

Project Coordinator:

Adarsh Gella

Erik Jonsson School of Engineering and Computer Science

University of Texas at Dallas

Date: May 10th, 2026

Table of Contents

Executive summary

1. Introduction

2. Project Specification & Management 3.
System Architecture & High-Level Design

4. Implementation & AI Orchestration 5.
System Performance & Validation 6.

Lessons Learned

7. Conclusion & Future Work

- References
- Acknowledgements

Executive Summary

Beyond Chat is a studio-based AI workspace developed at the University of Texas at Dallas, built around the principle that AI-generated outputs are most valuable when they are organized, reusable, and searchable — not buried in endless chat transcripts.

The product organizes work into eight purpose-built Studios (Chat, Writing, Research, Image, Data, Finance, Artifacts, and a unified Home dashboard), each with its own workflow, controls, and artifact-saving conventions. All outputs, from writing drafts to research reports to generated images, become artifacts that can be reopened, exported, and reused as context in any future studio run.

The technical stack is: React + TypeScript + Vite + Tailwind CSS on the frontend, FastAPI on the backend, Supabase for auth/database/storage, OpenRouter for multi-model access, Exa for live web research, and Stripe for billing, and everything is deployed on Vercel.

Key engineering accomplishments include: a fully auth-gated multi-studio product surface, real time streamed AI responses, a shared Compare Panel for side-by-side model evaluation, CSV/XLSX data analysis with AI-generated charts, multi-output Writing Studio launch kit generation, profile-scoped artifact ownership with row-level security in Postgres, and a 19-migration Supabase schema build-out. The app is live at beyond-chat-wheat.vercel.app with 148 commits across the development period.

1. Introduction

1.1 Motivation & Problem Statement

Modern AI assistants like ChatGPT and Claude are great at just generating content, but they share a fundamental flaw: every output, research report, or drafted article collapses into a scrolling chat transcript. Users have no way to search, version, or reuse prior outputs. Every new task forces a fresh prompt thus losing all prior context.

This forces knowledge workers into an copy-paste loop: generate in chat, manually transfer to a document tool, re-attach context for the next iteration. The product opportunity is a workspace that treats AI outputs as persistent artifacts from the moment they are created, structuring them by task type rather than order.

We chose this problem because our team directly experienced this issue during coursework and research. We believed a well-designed product could change how people interact with AI.

1.2 Objectives, Scope, and Goals Achieved

The project intended to deliver:

- A multi-studio AI workspace with Chat, Writing, Research, Image, Data, and Finance studios
- A unified Artifacts library with search, filtering, export, and handoff capabilities
- A shared Compare Panel for side-by-side model output comparison across any studio
- Supabase-backed auth, persistence, and storage with full row-level security
- A Stripe billing integration for free vs. Pro plan gating
- A live deployed product on Vercel

All core objectives were completed.

1.3 Proposed Solution

Beyond Chat replaces the generic chat surface with a suite of Studios: workflows for Chat, Writing, Research, Image generation, Data analysis, and Finance research. Every studio shares one app shell, one auth model, one artifact library, and one context model, thus outputs from any studio can be attached as context to runs in any other studio.

The key innovation is the Artifact, which is an exportable unit of AI-generated output. Instead of a message in a thread, every AI output in Beyond Chat is saved, tagged by studio type, linked to the run that produced it, and surfaced in a global Artifacts library. Users can filter by type, export to Markdown or PDF, bundle multiple artifacts, and hand off any artifact into another studio with its entire context preserved!

The Compare Panel, which is invocable from any studio rather than as a standalone route allows side-by-side model comparison without interrupting the active workflow, with results that are saveable as artifacts. The Finance Studio implements an agent-style tool-calling pattern

(inspired by a Dexter reference implementation) for structured financial research memos. The Data Studio accepts CSV/XLSX uploads, runs AI-powered analysis, and generates and saves charts, tables, and insight reports as separate artifacts.

1.4 Key Highlights

- Artifact-first architecture: every AI output is a first-class searchable/exportable artifact • 8 Studios shipped: Home, Chat, Writing, Research, Image, Data, Finance, Artifacts • Shared Compare Panel: invoke side-by-side model comparison from any studio without a route change
- Data Studio: CSV/XLSX ingestion, AI analysis, chart generation (bar/line/pie/scatter), separate artifact saves for charts, tables, and insight reports
- Writing Studio: targeted-edit mode, multi-output launch kit generation, context-aware AI drafts
- Finance Studio: Dexter-pattern tool-calling agent, structured timeline/memo output • OpenRouter multi-model routing with retry handling and parallel compare execution • Live deployment: beyond-chat-wheat.vercel.app (frontend + backend on Vercel)

2. Project Specification & Management

2.1 The Project Specification (The AI's Blueprint)

The project spec is in `spec.md` in the repository root. It served as the agent-ready instruction document outlining our AI-assisted development. The spec defines the product's surface area (public and protected routes), studio expectations with per-studio feature requirements, the technical contract (frontend stack, backend stack, persistence model), UX rules, non-goals, and the done-criteria for the reorganization phase.

Spec document: <https://github.com/KushagraBharti/Beyond-Chat/blob/main/spec.md>

2.2 Maintaining the 'Single Source of Truth'

Version control was maintained through Git on the main branch with 148 commits. The repository uses a layered documentation strategy to prevent spec drift:

- `spec.md` is the canonical product and UX contract — all studio feature requirements, route definitions, and non-goals live here
- `README.md` is the canonical stack and setup reference — conflicts with older notes are resolved in favor of `README.md` and `spec.md`
- `manual.md` captures all setup steps that cannot be automated in-repo (Supabase dashboard config, OAuth credentials, Stripe product creation)
- `completed.md` tracks every shipped feature with enough specificity for AI agents to understand current state without re-reading source
- `blocker.md` tracks unresolved external dependencies
- `backend/sql-related-files/` is the source of truth for the live Supabase schema, not local migration history

`CLAUDE.md` and `AGENTS.md` encode constraints specifically for AI coding agents (Claude Code and similar) to prevent agents from regenerating work that is already complete or from violating architectural decisions already made.

2.3 Key Milestones

The project progressed through these phases:

- Phase 1: Spec & Architecture: `spec.md` authored, system architecture documented, initial DB schema designed
- Phase 2: Auth & Backend Bootstrap: FastAPI setup, Supabase auth integration, JWT middleware, workspace bootstrap, health endpoint, artifact CRUD
- Phase 3: Frontend Studio Shell: React Router setup, Supabase client, auth callback routes, protected route surface, dashboard

- Phase 4: Studio Implementation: Chat, Writing, Research, Image, Data, Finance studios shipped with full artifact-save and handoff flows
- Phase 5: Artifact Library & Compare: shared Compare Panel, Artifacts studio with search/filter/export/bundle, cross-studio handoffs
- Phase 6: Schema Hardening: 19 Supabase migrations applied, RLS policies, billing tables, profile-scoped artifacts, security hardening (May 4, 2026)
- Phase 7: Deployment: Vercel deployment of both frontend and backend, production environment validation

2.4 Team Roles & Leader Rotation

Member	Leader Period
Kushagra Bharti	2/12-2/19
Yuvraj Kashyap	2/20-2/27
Kushagra Bharti	2/28-3/31
Diya Mehta	4/1-4/18
Harshwardhan Kothari	4/19-5/1

Team Responsibilities:

Kushagra Bharti: Agent workflow logic, tool runner/timeline, deployment support, frontend design

Yuvraj Kashyap: APIs, Openrouter Integration, APIs, deployment support Nishant

Bhagat: Supabase Schema, RLS security, artifact storage, Stripe integration Diya

Mehta: UI components, styling consistency, responsive design

Harshwardhan Kothari: Finance/Data Studios, Artifact Views, Compare feature

3. System Architecture & High-Level Design

3.1 System Overview

Beyond Chat is a two-tier web application: a React SPA frontend and a FastAPI Python backend, both deployed as separate Vercel projects from the same repository. All persistence is hosted on Supabase (Postgres + Storage). The frontend proxies all api requests to the backend during development and uses `VITE_API_BASE_URL` in production.

The backend exposes a RESTful API with JWT-authenticated endpoints. Supabase Auth issues JWTs on login/signup; the backend middleware validates JWTs against the Supabase JWKS or JWT secret and resolves workspace context from the authenticated user. All data operations are workspace-scoped with Postgres RLS policies enforced at the database level as a second security layer.

API documentation: https://github.com/KushagraBharti/Beyond-Chat/blob/main/docs/api_spec.md

3.2 Architecture Diagrams

High-Level Architecture:



Database schema: 19 Supabase migrations covering workspaces, members, chat threads/messages, artifacts, runs/run_steps, integrations, billing, storage policies, and RLS. Schema source: backend/sql-related-files/.

Architecture summary: <https://github.com/KushagraBharti/Beyond-Chat/blob/main/docs/system-architecture.md>

3.3 Key Architectural Decisions and Trade-offs

Supabase-only hosted runtime: we committed to Supabase for auth, database, and storage from the start, avoiding multi-vendor complexity. The trade-off is vendor lock-in, but the gain is a unified auth/data/storage contract that avoids a separate auth server, a separate file server, and multiple credential sets. A local SQLite bypass exists in the codebase as a legacy artifact but is not part of the product architecture.

OpenRouter for model access: rather than integrating individual model provider SDKs (Anthropic, OpenAI, etc.), we route all model calls through OpenRouter. This gives us single-key access to dozens of models and makes the Compare Panel far easier to implement. Shared Compare Panel vs. standalone route: the Compare feature was intentionally de-routed from a standalone /compare destination to a shared panel invocable from Chat, Writing, Research, and Finance. This eliminated duplicate UI surface and forced Compare outputs to live inside the artifact-producing context of the active studio.

4. Implementation & AI Orchestration

4.1 Technologies Used

Technology	Layer	Purpose
React 18 + TypeScript	Frontend	Component model, type safety, SPA routing
Vite	Frontend	Build tool, hot module replacement, dev proxy
Tailwind CSS	Frontend	Utility-first styling across all studio surfaces
React Router v6	Frontend	Client-side routing for public and protected routes
FastAPI	Backend	Python API framework, async request handling, OpenAPI docs
uv	Backend	Python package/env manager (pip not used)
Supabase Auth	Auth	JWT issuance, OAuth callbacks, session management
Supabase Postgres	Database	All relational data; RLS enforced at DB level
Supabase Storage	Storage	Artifact files, user-uploads (CSV/XLSX), generated images
OpenRouter	AI Routing	Multi-model LLM access: chat, compare, research, finance, writing, image
Exa Search	Research	Live web source gathering for Research and Finance studios
Stripe	Billing	Subscription checkout, billing portal, webhook handling
Vercel	Deployment	Frontend (Vite preset) + backend (Other preset) as two separate projects
Vitest	Testing	Frontend unit/smoke tests
pytest	Testing	Backend API test suite

4.2 AI Build Strategy

The development process was heavily AI-orchestrated using Claude Code (command-line agentic coding) guided by a structured set of project documents. The core AI build strategy had three components:

1. Agent-ready specification documents: `spec.md`, `CLAUDE.md`, `AGENTS.md`, and `completed.md` served as the master context injected into every Claude Code session. `CLAUDE.md` encodes hard constraints (e.g., never use `pip`, always use `uv`; never break existing passing tests; never modify the Supabase schema without explicit instruction). `AGENTS.md` encodes agentic workflow rules for multi-step tasks.
2. Completed work tracking: `completed.md` was updated after every significant implementation session, giving AI agents a precise record of what exists so they do not regenerate work or create conflicting implementations. Each entry is specific enough to describe the data flow, not just the feature name.
3. Prompt library for in-product AI features, each studio uses a distinct system prompt pattern. The Research Studio sends a structured prompt requesting competitor/landscape and opportunity/risk matrices with explicit source-limit handling. The Finance Studio implements a Dexter-inspired tool-calling agent pattern. The Writing Studio targeted-edit mode sends selected text plus bounded surrounding context. These prompt strategies are documented inline in the backend source.

Prompt library and agent specs: <https://github.com/KushagraBharti/Beyond-Chat/blob/main/CLAUDE.md>

Agent rules: <https://github.com/KushagraBharti/Beyond-Chat/blob/main/AGENTS.md>

4.3 Human-in-the-Loop Interventions

The following interventions required manual developer action to resolve:

- Supabase CLI push blocked by remote migration-history drift: The CLI circuit breaker on `cli_login_postgres` blocked schema pushes. Resolution: applied all pending migrations directly through Supabase MCP/SQL editor rather than via CLI, then mirrored versions locally.
- Supabase schema wipe and rebuild: After RLS policy drift introduced helper-function auth breaks, the disposable hosted project was intentionally wiped and the canonical SQL in `backend/sql-related-files/` was re-applied from scratch, followed by `direct_membership_rls` migration to repair the bootstrap flow.
- SECURITY DEFINER RPC vulnerability: AI-generated Supabase RLS scaffold left public SECURITY DEFINER functions executable by the authenticated role. Manual intervention added migration

20260504063424_revoke_authenticated_security_definer_rpc.sql and moved workspace bootstrap to the service-role client.

- OpenRouter tool-call schema conflicts: some models returned tool_call blocks in unexpected formats. Manual override added response normalization to extract text blocks from mixed content arrays before rendering in the Compare Panel.
 - Fake/seeded dashboard data removal: early AI-generated dashboard code included hardcoded provider tiles and demo events. Manual pass removed all deterministic fake data and replaced with real normalized provider statuses and a live artifact activity panel.
- frontend-mock/ archival: the AI agents initially continued writing into frontend-mock/ as if it were an active product surface. A manual CLAUDE.md update explicitly flagged that directory as archived reference material, not an active application.

4.4 Codebase

GitHub Repository: <https://github.com/KushagraBharti/Beyond-Chat>

README.md: <https://github.com/KushagraBharti/Beyond-Chat/blob/main/README.md> API

Specification: <https://github.com/KushagraBharti/Beyond-Chat/blob/main/docs/api-spec.md>

Live Application: <https://beyond-chat-wheat.vercel.app>

Repository structure:

- frontend/ — React + TypeScript + Vite production frontend
- backend/ — FastAPI production API and AI orchestration runtime
- backend/sql-related-files/ — canonical Supabase/Postgres schema (source of truth) •
- docs/ — api-spec.md, system-architecture.md, sprint planning
- weeklyUpdates/ — team progress log by week
- spec.md / manual.md / completed.md / blocker.md — living project docs •
- CLAUDE.md / AGENTS.md — AI agent constraint and workflow files

5. System Performance & Validation

5.1 Testing and Validation

Backend testing uses pytest via `uv run pytest`. The test suite covers the health endpoint, auth bootstrap, workspace resolution, chat CRUD, artifact CRUD, run creation and step recording, export, and integration status endpoints. Tests are structured to run against a local SQLite backed store (`store.py`) for isolation without requiring a live Supabase connection.

Frontend testing uses Vitest with a setup for smoke tests of core component rendering. Validation also includes `npm run lint` (ESLint) and `npm run build` (Vite production build). API contract testing was performed with Postman against the local FastAPI server during development.

Browser QA was completed locally against the backend-served frontend build for: login bypass, dashboard rendering, chat compare mode, writing library and editor navigation, research error state handling, artifact export, and settings/provider status rendering.

5.2 Technical Performance Metrics

5.3 AI Model Performance

Research Studio: live Exa-backed runs are required; the backend fails explicitly when Exa is missing rather than generating fallback research. This makes research output quality directly measurable by source count and citation accuracy.

Compare Panel: parallel OpenRouter calls execute simultaneously for both models. Tool-call responses are normalized to extract text blocks from mixed content arrays, preserving requested tool call structure for supported models.

Finance Studio: the Dexter-pattern tool-calling agent produces structured timeline and memo outputs. Run steps are recorded individually, giving full step-history visibility per run.

6. Lessons Learned

6.1 Insights Gained

The most significant shift was moving from code-first to spec-first development. In traditional engineering, you write code and update documentation afterward. In this project, the spec.md, CLAUDE.md, and completed.md had to be written prior to the code because the AI agents needed accurate context to generate output. A stale spec produced a lot of stale or conflicting code.

We also learned that AI agents are very good at implementing a well-defined contract and very bad at inferring unstated architectural decisions. Every time we left a gap in the spec the agents filled it in the wrong direction. The fix was always documentation and not prompting.

6.2 Lessons from Challenges

- Supabase CLI migration drift: pushing schema changes with the CLI becomes unreliable once remote migration history diverges from local. Resolution: treat backend/sql-related files/ as the source of truth and apply migrations via SQL editor/MCP when CLI is blocked. Never treat supabase/migrations/ as authoritative without cross-referencing.
- RLS policy drift: AI-generated RLS scaffolds drifted from the actual runtime security model as the schema evolved. Resolution: dedicated security-hardening migrations to revoke unsafe RPC permissions, and manual review of the Supabase security advisor after each schema change.
- Fake data in AI-generated UIs: agents tend to scaffold demo/seed data to make UIs look functional. This caused the dashboard to show hardcoded provider tiles and fake calendar events. Resolution: explicit CLAUDE.md rule: never use hardcoded demo data; all UI state must come from real API responses or explicit unavailable states.

6.3 Skills Developed

The team developed lots of practical skills in AI-orchestrated engineering: writing agent-ready specifications, maintaining living documentation as a development artifact, and using completed.md as a form of machine-readable progress tracking. We also developed depth in Supabase (RLS policy design, migration management, storage bucket policies), FastAPI (JWT middleware, async endpoints, Vercel serverless deployment), and React architecture (feature oriented structure, shared UI primitives, Supabase client integration).

6.4 Individual Reflections

Kushagra Bharti

Working on Beyond Chat taught me a lot about how important planning and organization are when building a large AI project. I spent a lot of time working on the agent workflows, frontend design, and deployment support, and I realized pretty quickly that even small architectural decisions could affect the entire application later on. One of the biggest things I learned was that AI tools work best when the project documentation is really clear, because unclear specs usually led to confusing or broken implementations.

Nishant Bhagat

This project gave me more experience with backend systems and database management. I worked mainly on the Supabase schema, storage, Stripe integration, and security policies, which helped me better understand how real applications handle user data and permissions. I also learned that security and database structure become much harder to manage as a project grows, especially when multiple people are constantly making changes.

Yuvraj Kashyap

Through this project, I got more experience working with APIs, OpenRouter integration, and deployment issues. A lot of my work involved making sure the frontend and backend could communicate reliably with different AI models without breaking. I learned that debugging deployment and API issues can sometimes take longer than building the actual feature, but those problems are a huge part of real-world software development.

Harshwardhan Kothari

Working on the Finance Studio, Data Studio, Artifact Views, and Compare feature helped me learn more about building features that feel useful instead of just technically functional. I spent a lot of time thinking about how users would interact with AI-generated outputs and how to organize them in a clean way. One thing I learned from this project is that building a good user experience takes just as much effort as building the backend functionality itself.

Diya Mehta

This project helped me improve a lot in frontend development and UI design. I focused mostly on styling consistency, reusable UI components, and making sure the app looked clean across different pages and screen sizes. I learned how important small design details are in making a large application feel connected and easier to use, especially when there are many different features and workflows all inside one product.

7. Conclusion & Future Work

7.1 Summary of Key Accomplishments

Beyond Chat successfully shipped an artifact-first AI workspace with eight purpose-built studios, a shared Compare Panel, a searchable Artifacts library, and a full Supabase-backed persistence and auth model — all deployed live on Vercel. The product's done criteria (from spec.md) were fully met: the protected route surface matches the spec; Compare is only available through shared invocation; the codebase presents one canonical stack and runtime story; all major studios preserve artifact context through their main handoff paths.

7.2 Future Enhancements

- Collaborative workspaces: the schema already has workspace membership tables and workspace-scoped storage paths; activating multi-user collaboration within a workspace is a natural next phase
- Fine-grained billing and usage tracking: the billing schema tracks usage events; a future version could implement per-studio usage metering and dynamic Pro tier limits
- Artifact versioning: current artifacts are point-in-time snapshots; adding version history (linked to the run lineage already recorded) would make the Writing Studio suitable for long-form iterative document work
- Mobile-responsive studio surfaces: current studios are designed for desktop; a focused mobile layout for Chat and Artifacts would expand the addressable use case

References

- Supabase Documentation — <https://supabase.com/docs>
- FastAPI Documentation — <https://fastapi.tiangolo.com>
- OpenRouter API Documentation — <https://openrouter.ai/docs>
- Exa Search API — <https://exa.ai/docs>
- Stripe API Reference — <https://stripe.com/docs/api>
- React Documentation — <https://react.dev>
- Vite Documentation — <https://vitejs.dev>
- Tailwind CSS Documentation — <https://tailwindcss.com/docs>
- Vercel Deployment Documentation — <https://vercel.com/docs>
- Beyond Chat Repository — <https://github.com/KushagraBharti/Beyond-Chat>

Acknowledgements

The team thanks Professor Sridhar Alagar for guidance and oversight throughout the course, and Project Coordinator Adarsh Gella for great support and feedback across the development milestones. We also acknowledge the open-source maintainers of FastAPI, Supabase, React, Vite, Tailwind CSS, and the broader ecosystem on which Beyond Chat is built.